Income Prediction in Brazil: A ML Approach

Utilizing census microdata from Brazil, can we apply ML techniques to predict income?

João Martins

MACS 33002 – Intro to ML

Professor Zhao Wang

Motivation and Research Question

Literature Overview

Literature shows income prediction is feasible using simple features across different contexts:

- Mexico study: Census microdata with tree-based ML models to predict poverty
- World Bank study: Simple regression predicting income distributions across countries
- Spatial data study: Satellite and street view imagery to model household income

Why Income Tracking Matters

Income tracking is essential for designing poverty reduction policies and understanding economic development

Initial Research Question: Utilizing census microdata from Brazil, can we predict household income b
a handful of simple features?

Final Research Question: Utilizing census microdata from Brazil, can we apply ML techniques to
income?

Data Acquisition & Task Formulation

Data Source

- PNAD Contínua (*Pesquisa Nacional por Amostra de Domicílios Contínua*)
- Brazilian Institute of Geography and Statistics (IBGE)
- Household survey with ~400 features covering demographics, education, employment, housing, and income

Task Formulation

- Task Type: Regression (predict continuous HH income)
- Target Variable: Monthly household income (R$)
- Feature Selection: Use Lasso regression for feature selection, then apply selected features to Random model
- Evaluation Metrics: RMSE, MAE, R^2

Data Processing

Very heavy dataset, so these were my first steps:

1. Loaded 2025 Q2 PNAD Contínua microdata with pnadium
2. Dropped IDs and income-like variables to prevent leakage
3. Kept adults (age ≥ 18) with non-missing income → Sample size ≈ 200k, 392 features
4. Filled remaining predictor NAs with 0 when consistent with coding
5. Split into train/validation/test; standardized features for Lasso only

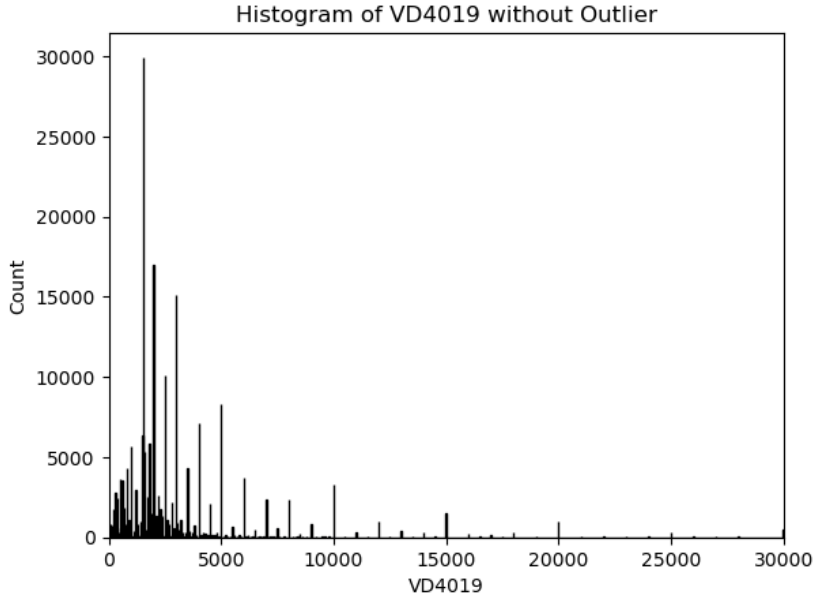
```
df
```

	UF	V1016	V1022	V1023	V1027	V1028	V1029	V1033	posest	posest_sxi	...	V1028191	V1028192	V1028193	V1028194	V1028195	V
2	11	3	1	1	299.388679	370.470690	517823	6286664	111	111	...	0.000000	0.0	386.052699	364.309477	388.448283	(
3	11	3	1	1	299.388679	370.470690	517823	7979836	111	107	...	0.000000	0.0	386.052699	364.309477	388.448283	(
6	11	3	1	1	299.388679	383.565718	517823	8310816	111	208	...	0.000000	0.0	401.601780	377.005854	402.832840	(
7	11	3	1	1	299.388679	383.565718	517823	8310816	111	208	...	0.000000	0.0	401.601780	377.005854	402.832840	(
8	11	3	1	1	299.388679	383.565718	517823	8019965	111	108	...	0.000000	0.0	401.601780	377.005854	402.832840	(
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
475581	53	4	2	1	193.675834	228.860209	2996947	7334313	531	110	...	234.215635	0.0	254.799161	241.187607	768.879212	25
475582	53	4	2	1	193.675834	228.860209	2996947	6866767	531	211	...	234.215635	0.0	254.799161	241.187607	768.879212	25
475585	53	4	2	1	193.675834	243.389427	2996947	7334313	531	110	...	254.837224	0.0	263.761673	246.723934	795.845198	26
475586	53	4	2	1	193.675834	243.389427	2996947	7921315	531	105	...	254.837224	0.0	263.761673	246.723934	795.845198	26
475589	53	4	2	1	193.675834	181.038434	2996947	3907408	531	114	...	193.952705	0.0	202.032884	181.202196	593.147235	19

Exploratory Data Analysis

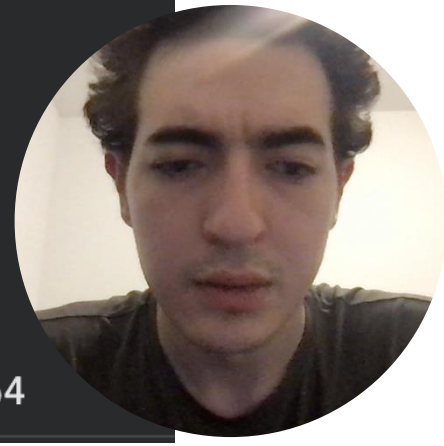
Target Variable Distribution (VD4019 = Monthly Income)

- Removed extreme outliers (monthly income >R$30,000, approximately 6 standard deviations) to improve model stability
- Income data is heavily right-skewed (even after outlier removal), as shown in the summary statistics
- Mean income (R$3,062) substantially exceeds median (R$2,000), indicating concentration at lower values with long right tail




```
df["VD4019"].describe()

count    204469.000000
mean       3062.045259
std        3461.922271
min          10.000000
25%        1518.000000
50%        2000.000000
75%        3200.000000
max       30000.000000
Name: VD4019, dtype: float64
```



Linear Model (Lasso Regression)

Why Lasso?

- Automatic feature selection through L1 regularization (shrinks coefficients to zero)
- Simple, interpretable linear model establishes baseline performance

Pros

- Built-in features selection, which reduces overfit risk
- Works well when many predictors are irrelevant
- Coefficients easy to interpret

Cons

- Unstable when predictors are highly correlated
- Linear structure may miss non-linear relationships
- Requires careful tuning of α: too large over-shrinks coefficients, too small overfits

Formula

$$= \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \underbrace{\|y - X\beta\|_2^2}_{\text{Loss}} + \lambda \underbrace{\|\beta\|_1}_{\text{Penalty}}$$

Linear Model (Cont.) – Parameter Tuning and Performance

Feature Selection Process

- Started with 392 features from census microdata
- Tested $\alpha = \{1, 5, 10\}$; max_iter = 20,000; optimal $\alpha = 1$ (best validation R^2)
- Lasso selected **298 non-zero features** from original 392
- These 298 features fed into Random Forest model for improved prediction

Model Performance ($\alpha = 1$, max_iter=20,000) - Validation

$R^2 = 0.457$ | MAE = R$1,454 | RMSE = R$2,547

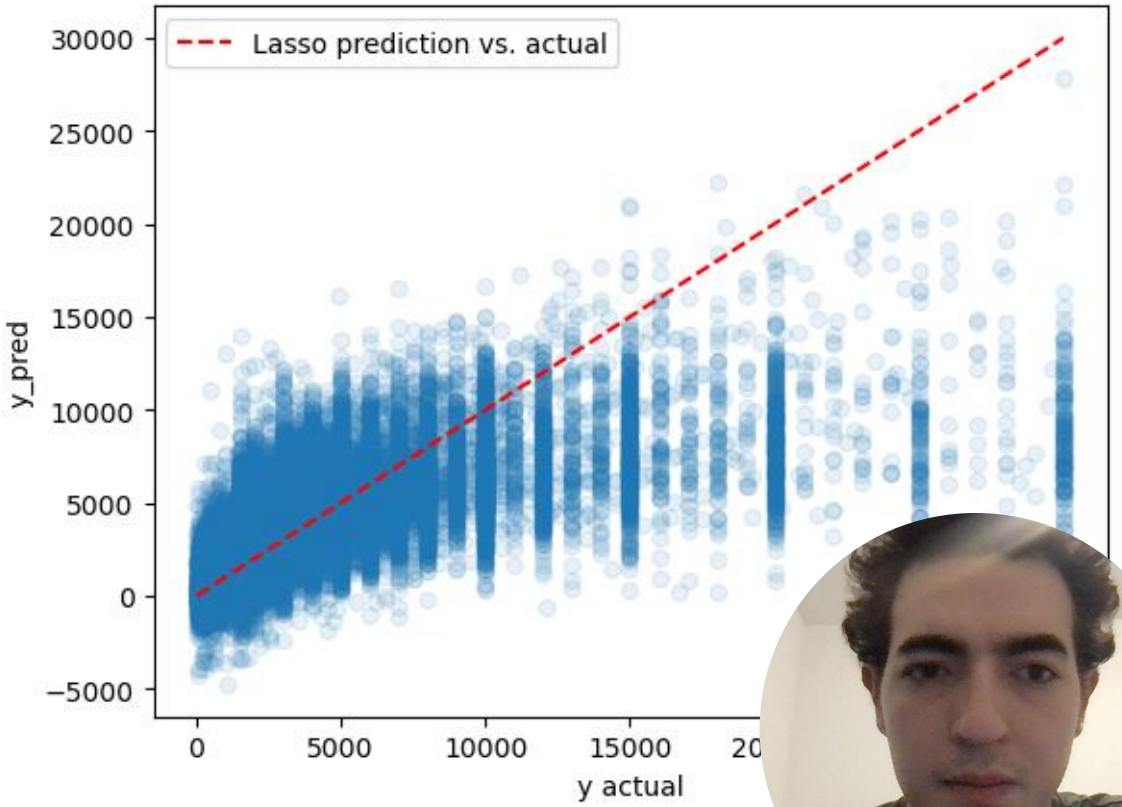
Linear Model (Cont.) – Error Analysis and Coefficients

Top 10 features by coefficient

```
importance_df = pd.DataFrame({
    "feature": X.columns,
    "coef": lasso_1.coef_
}).sort_values("coef", key=abs, ascending=False)

top_features = importance_df[importance_df.coef != 0]
print(top_features.head(10))
```

```
      feature         coef
23     V3003A  2147.027131
32     V3009A  2043.566917
175    VD4009  2014.422309
60      V4014 -1607.703869
61      V4015 -1146.437121
121  V405111   950.095730
36      V3012  -881.255661
55      V4012   859.335721
76      V4019  -751.066704
69      V4017  -647.044437
```



Tree-Based Model (Random Forest)

How Random Forest Works

- Builds many decision trees on bootstrapped data samples
- Each tree splits on random subsets of features
- Final prediction averages across all trees

Pros

- Strong predictive performance on complex tabular data
- Automatic interaction modeling
- Tolerant of irrelevant features
- Feature importance rankings

Why Random Forest?

- Captures non-linearities and interactions
- Ensemble reduces variance compared to single decision tree
- Best-performing model in many income-prediction papers

Cons

- Hard to interpret (black box)
- Computationally and memory intensive
- Poor extrapolation beyond training range
- Unstable with extreme outliers

Tree-Based Model – Hyperparameter Tuning and Performance

Best Parameters (RandomizedSearchCV, 12 iterations, 3-fold CV)

- n_estimators: 300 trees
- max_depth: None
- min_samples_split: 2
- min_samples_leaf: 5
- max_features: 'sqrt'

```
print('Best params:', rf_random.best_params_)
best_rf = rf_random.best_estimator_
y_val_pred = best_rf.predict(X_val)
y_test_pred = best_rf.predict(X_test)

Best params: {'n_estimators': 300, 'min_samples_split': 2, 'min_samples_leaf': 5, 'max_features': 'sqrt', 'max_depth': None}
```

Model Performance (Validation)

$R^2 = 0.519$ | MAE = R$1,294 | RMSE = R$2,429

Note

- Features selected by Lasso (298 non-zero coefficients) fed into Random Forest
- MAE < RMSE confirms sensitivity to outliers remains

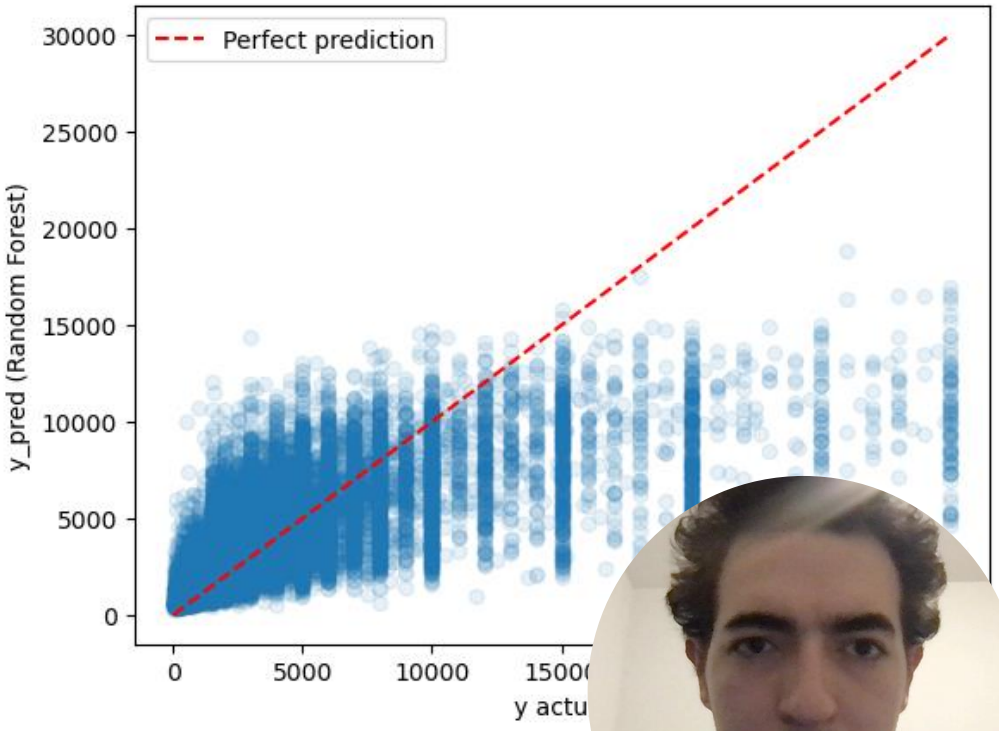
Tree-Based Model – Error Analysis and Coefficients

Most Important Features of Random Forest Model

```
portances = best_rf.feature_importances_
_importance_df = pd.DataFrame({"feature": X.columns,"importance":

int(rf_importance_df.head(10))
```

	feature	importance
36	V4010	0.059973
109	VD3004	0.047639
110	VD3005	0.045045
111	VD3006	0.036337
24	V3009A	0.036296
115	VD4011	0.035513
78	V40403	0.021988
46	V4016	0.016921
93	V405111	0.016060
116	VD4012	0.014613



Model Performance (Test)

	Lasso Regression	Random Forest
R^2	0.456	0.515
MAE	R$1,451	R$1,269
RMSE	R$2,531	R$2,391

Takeaways

- Using 298 selected census features, the model explains about **52% of the variance in household income**
- Random Forest outperforms linear Lasso regression on both validation and test sets, indicating important non-linearities in the income process
- The model performs well around the middle of the distribution but struggles at the extremes (very poor and very rich households)
- **From a policy perspective, education, occupation, and formal labor market attachment emerge as important predictors of income, reinforcing the importance of skills, schooling quality, and formal business/job creation in raising incomes**
- Overall, this is a great proof of concept that income prediction is feasible and useful for policy design and anti-poverty policies

Possible Improvements

- Model is sensitive to outliers and the heavy-tailed income distribution
- Could reframe the problem as a classification task (e.g., income brackets, deciles) to reduce the impact of extreme values and focus on correctly assigning households to policy-relevant bins
- Another avenue is k-means on residuals to identify segments where the model systematically under- or over-predicts
- More systematic model comparison could further improve predictive accuracy over the current Random Forest baseline